

UNIVERSIDAD RAFAEL LANDIVAR

Facultad de Ingenieria

Ingenieria en Sistemas

LABORATORIO No. 1 - SEMANA #13

APIs con Swagger & Postman

Curso:	Manejo e Implementacion de Archivos (MIA)
---------------	---

Estudiante:	Michael Gutierrez
--------------------	-------------------

Carne:	11997
---------------	-------

Fecha:	2 de octubre de 2026
---------------	----------------------

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, metodos HTTP, parametros, cuerpos JSON, codigos de respuesta y operaciones CRUD.

Investigacion previa

¿Que es una API REST?

Una API REST es una interfaz que permite que diferentes aplicaciones se comuniquen mediante solicitudes HTTP. Organiza la informacion como recursos y permite consultarlos, crearlos, modificarlos o eliminarlos mediante operaciones estandar.

¿Que es un endpoint?

Un endpoint es una ruta especifica de una API utilizada para acceder a un recurso o ejecutar una operacion. Por ejemplo, `/pet/11997` representa el acceso a una mascota identificada con ese ID.

¿Que diferencia existe entre una URL y un endpoint?

Una URL es una direccion completa que identifica una ubicacion en la web. Un endpoint es la ruta concreta dentro de una API que atiende una operacion determinada. Por ejemplo, la URL completa puede ser `https://petstore3.swagger.io/api/v3/pet/11997`, mientras que el endpoint es `/pet/{petid}`.

¿Que funcion cumplen los metodos HTTP GET, POST, PUT/PATCH y DELETE?

GET consulta informacion. **POST** crea un recurso. **PUT** actualiza un recurso completo y **PATCH** modifica parcialmente algunos campos. **DELETE** elimina un recurso.

¿Que es JSON y por que es comun en las APIs?

JSON (JavaScript Object Notation) es un formato de texto para representar datos mediante pares de clave y valor. Es comun en APIs porque es ligero, legible y compatible con numerosos lenguajes de programacion.

¿Que es un codigo de estado HTTP? Mencione ejemplos de 200, 404 y 500.

Un codigo de estado HTTP informa el resultado de una solicitud. **200** indica que la operacion fue exitosa; **404** indica que el recurso solicitado no fue encontrado; y **500** corresponde a un error interno del servidor.

¿Que informacion se puede encontrar en la respuesta de una API?

Una respuesta puede incluir el codigo de estado, encabezados (headers), un cuerpo con datos normalmente en JSON y, cuando existe un problema, mensajes que describen el error.

¿Que ventajas tiene utilizar Postman para probar una API antes de programarla?

Postman permite probar manualmente solicitudes HTTP, configurar parametros, headers y cuerpos JSON, observar respuestas y detectar errores antes de integrar la API en una aplicacion.

¿Que es y para que sirve Swagger UI?

Swagger UI es una interfaz interactiva que documenta los endpoints de una API y permite probarlos desde el navegador, mostrando metodos, parametros, cuerpos esperados y respuestas.

¿Que funcion cumple Postman en el desarrollo y prueba de APIs?

Postman permite crear, guardar, organizar y ejecutar solicitudes contra una API. Facilita validar operaciones CRUD, revisar codigos HTTP, comparar respuestas y conservar colecciones de pruebas.

1. Preparacion del entorno

pet Everything about your Pets		Find out more ^
PUT	/pet Update an existing pet.	🔒 ▼
POST	/pet Add a new pet to the store.	🔒 ▼
GET	/pet/findByStatus Finds Pets by status.	🔒 ▼
GET	/pet/findByTags Finds Pets by tags.	🔒 ▼
GET	/pet/{petId} Find pet by ID.	🔒 ▼
POST	/pet/{petId} Updates a pet in the store with form data.	🔒 ▼
DELETE	/pet/{petId} Deletes a pet.	🔒 ▼
POST	/pet/{petId}/uploadImage Uploads an image.	🔒 ▼

Evidencia 1. Endpoints disponibles en la seccion pet de Swagger Petstore 3.

Se utilizo Swagger Petstore 3 para identificar las operaciones relacionadas con **/pet**. Se localizaron los metodos PUT, POST, GET y DELETE necesarios para realizar las operaciones CRUD del laboratorio.

2. Exploracion inicial - Consulta por ID

The screenshot shows a REST client interface with the following sections:

- Curl:** A code block containing the command: `curl -X 'GET' \ 'https://petstore3.swagger.io/api/v3/pet/10' \ -H 'accept: application/json'`
- Request URL:** A text field containing: `https://petstore3.swagger.io/api/v3/pet/10`
- Server response:** A section with a status of **200**.
- Response body:** A JSON object: `{ "id": 10, "name": "Rufus", "photoUrls": [ "example.com" ], "tags": [] }`
- Response headers:** A list of headers: `access-control-allow-headers: Content-Type,api_key,Authorization`, `access-control-allow-methods: GET,POST,DELETE,PUT`, `access-control-allow-origin: *`, `access-control-expose-headers: Content-Disposition`, `content-length: 62`, `content-type: application/json`, `date: Fri,02 Oct 2026 22:25:20 GMT`, and `server: Jetty(9.4.53.v20231009)`
- Responses:** A table with the following data:

Code	Description	Links
200	successful operation	No links

Evidencia 2. Consulta inicial mediante GET /pet/{petId}.

Se utilizo el metodo **GET** sobre el endpoint **/pet/{petId}**, enviando el parametro **petId = 10**. La API respondio con **HTTP 200** y devolvio en formato JSON la informacion de la mascota con ID 10, llamada **Rufus**. Esto confirma que el endpoint consulta un recurso individual a partir de su identificador.

3. Crear un recurso con POST

Responses

Curl

```
curl -X 'POST' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 11997,
    "name": "MascotaMichael",
    "category": {
      "id": 11997,
      "name": "Perro"
    },
    "photoUrls": [
      "https://example.com/mascota11997.jpg"
    ],
    "tags": [
      {
        "id": 11997,
        "name": "MIA"
      }
    ],
    "status": "available"
  }'
```

Request URL

https://petstore3.swagger.io/api/v3/pet

Server response

Code	Details
200	Response body<pre>{ "id": 11997, "category": { "id": 11997, "name": "Perro" } }</pre>

Evidencia 3. Creacion de la mascota con ID 11997 mediante POST /pet.

Se creo una mascota propia utilizando **POST /pet**. El cuerpo JSON incluyo el ID **11997**, nombre **MascotaMichael**, categoria **Perro**, etiqueta **MIA**, una URL de fotografia y el estado **available**. El servidor respondio con **HTTP 200**, indicando que la solicitud fue procesada correctamente.

3.1 Verificación del recurso creado

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/11997' \
  -H 'accept: application/json'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet/11997
```

Server response

Code	Details
200	Response body <pre>{ "id": 11997, "category": { "id": 11997, "name": "Perro" }, "name": "MascotaMichael", "photoUrls": ["https://example.com/mascota11997.jpg"], "tags": [{ "id": 11997, "name": "MIA" }], "status": "available" }</pre> Download

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 184
```

Evidencia 4. Verificación de la mascota 11997 mediante GET.

Después del POST se ejecutó **GET /pet/11997**. La respuesta fue **HTTP 200** y mostró los mismos datos enviados, incluyendo el nombre **MascotaMichael** y el estado **available**. Con esta consulta se comprobó que el recurso podía recuperarse correctamente.

4. Modificar el recurso con PUT

Responses

Curl

```
curl -X 'PUT' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 11997,
    "name": "MascotaMichaelActualizada",
    "category": {
      "id": 11997,
      "name": "Perro"
    },
    "photoUrls": [
      "https://example.com/mascota11997.jpg"
    ],
    "tags": [
      {
        "id": 11997,
        "name": "MIA"
      }
    ],
    "status": "pending"
  }'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet
```

Server response

Code	Details
200	Response body <pre>{ "id": 11997,</pre>

Evidencia 5. Actualizacion de nombre y estado mediante PUT /pet.

Para actualizar la mascota se utilizo **PUT /pet**. Se conservaron el ID y los demas datos, pero se modificaron al menos dos atributos: el nombre cambio a **MascotaMichaelActualizada** y el estado cambio de **available** a **pending**. La operacion devolvio **HTTP 200**.

4.1 Comprobacion de la modificacion

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/11997' \
  -H 'accept: application/json'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet/11997
```

Server response

Code	Details
200	Response body <pre>{ "id": 11997, "category": { "id": 11997, "name": "Perro" }, "name": "MascotaMichaelActualizada", "photoUrls": ["https://example.com/mascota11997.jpg"], "tags": [{ "id": 11997, "name": "MIA" }], "status": "pending" }</pre> Download

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 193
```

Evidencia 6. GET posterior al PUT.

Se ejecuto nuevamente **GET /pet/11997**. La respuesta **HTTP 200** mostro el nombre **MascotaMichaelActualizada** y el estado **pending**, comprobando que los cambios fueron aplicados.

5. Consulta utilizando parametros - available

The screenshot displays a REST client interface with the following sections:

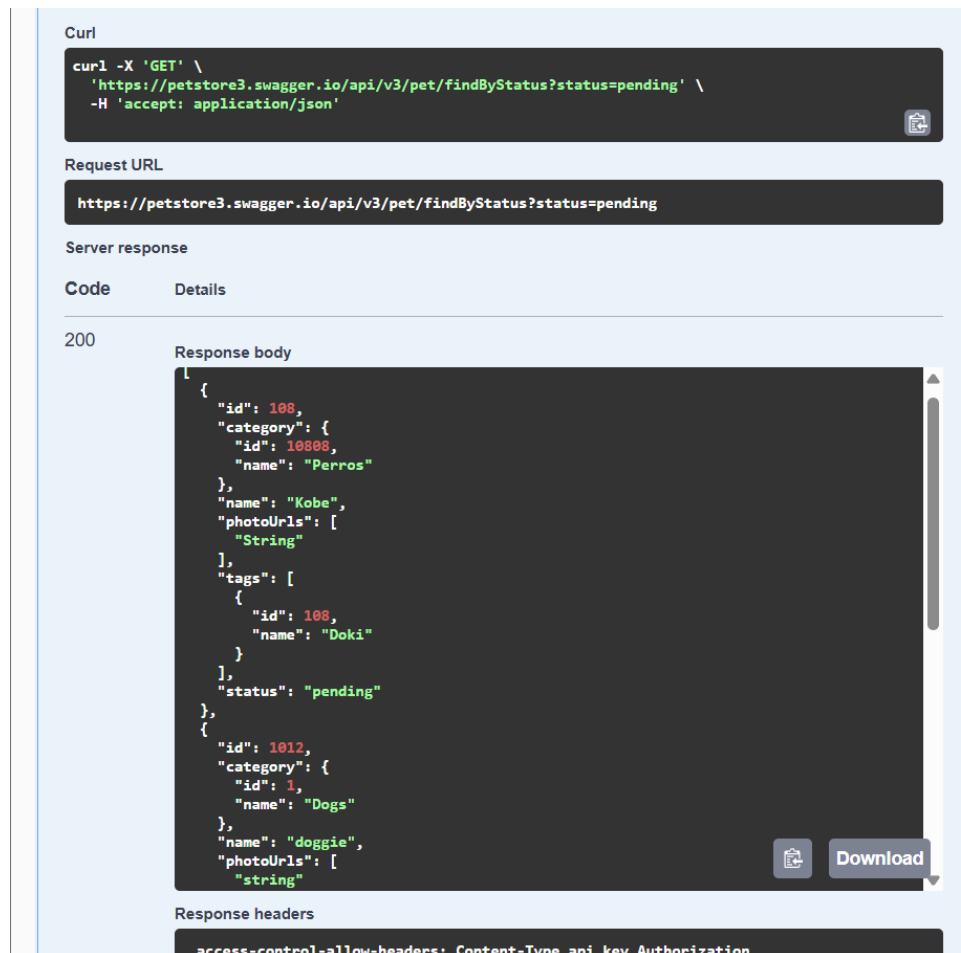
- Curl:** A dark box containing the command: `curl -X 'GET' \ 'https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available' \ -H 'accept: application/json'`
- Request URL:** A dark box containing the URL: `https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available`
- Server response:** A section with two tabs: **Code** (selected) and **Details**.
- Code:** Shows the status code **200**.
- Response body:** A dark box containing a JSON array of two pet objects. The first object has an ID of 12345, is a dog named 'fluffy', and has two tags: 'tag1' and 'tag2'. The second object has an ID of 123456 and is a dog named 'Dogs'. Both have an available status.
- Response headers:** A section at the bottom, currently empty.

A **Download** button is visible at the bottom right of the response body.

Evidencia 7. GET /pet/findByStatus con status=available.

Se utilizo **GET /pet/findByStatus** con el parametro de consulta **status=available**. El parametro se envio en la URL como query parameter. La API respondio con **HTTP 200** y un arreglo JSON que contiene mascotas cuyo estado es available.

5.1 Consulta utilizando parametros - pending



The screenshot displays a REST client interface with the following sections:

- Curl:** Shows the command `curl -X 'GET' \ 'https://petstore3.swagger.io/api/v3/pet/findByStatus?status=pending' \ -H 'accept: application/json'`.
- Request URL:** Shows the URL `https://petstore3.swagger.io/api/v3/pet/findByStatus?status=pending`.
- Server response:** Shows a **Code** of 200 and a **Response body** containing a JSON array of two pet objects. The first object is a cat named "Kobe" with status "pending", and the second is a dog named "doggie".
- Response headers:** Shows the header `access-control-allow-headers: Content-Type,api_key,Authorization`.

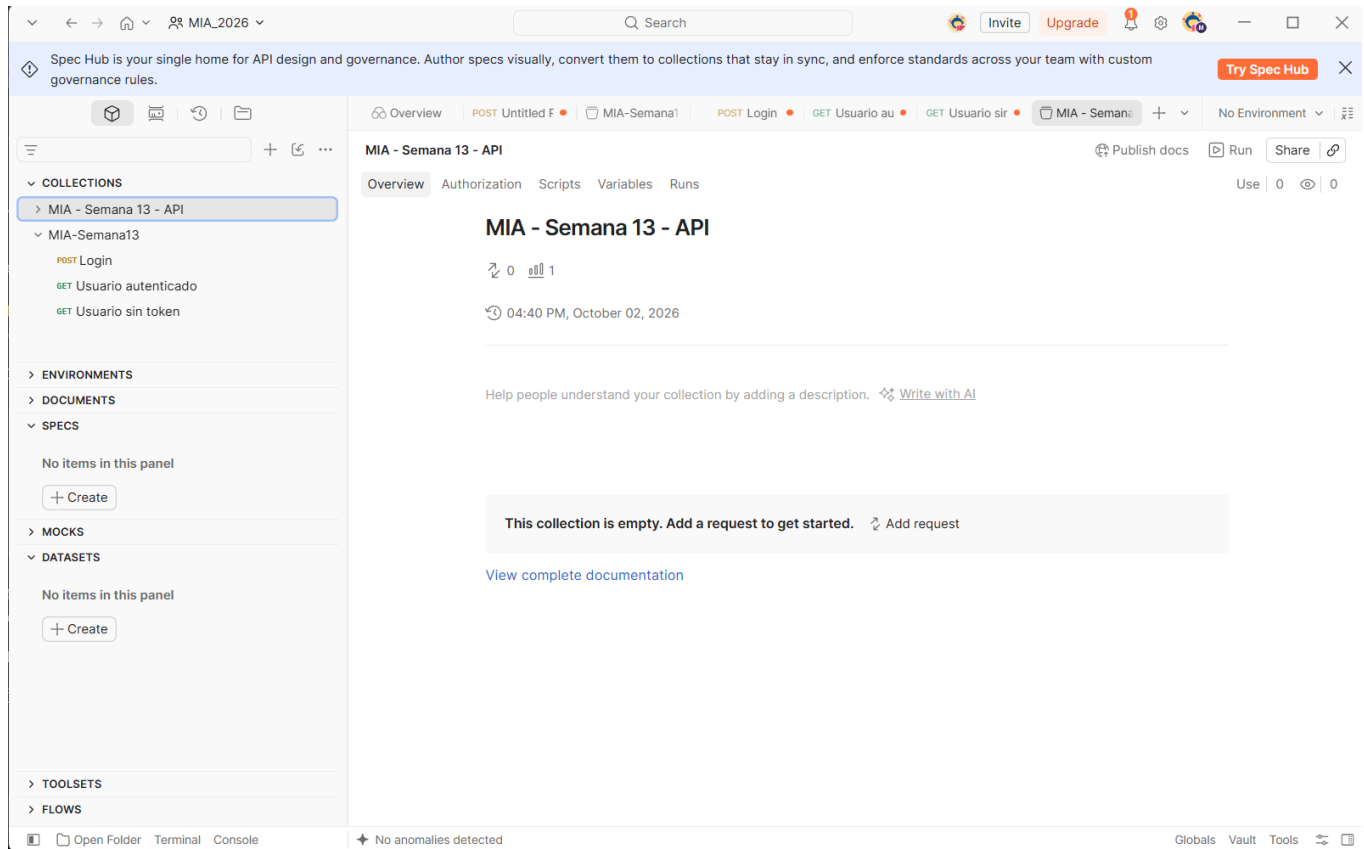
The JSON response body is as follows:

```
{
  "id": 1008,
  "category": {
    "id": 10000,
    "name": "Perros"
  },
  "name": "Kobe",
  "photoUrls": [
    "String"
  ],
  "tags": [
    {
      "id": 1008,
      "name": "Doki"
    }
  ],
  "status": "pending"
},
{
  "id": 1012,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "doggie",
  "photoUrls": [
    "string"
  ]
}
```

Evidencia 8. GET /pet/findByStatus con status=pending.

Al cambiar el valor del parametro a **pending**, la estructura de la respuesta se mantuvo como un arreglo JSON, pero el conjunto de resultados correspondio a mascotas con ese estado. Esto demuestra que el query parameter modifica el criterio de busqueda sin cambiar el endpoint principal.

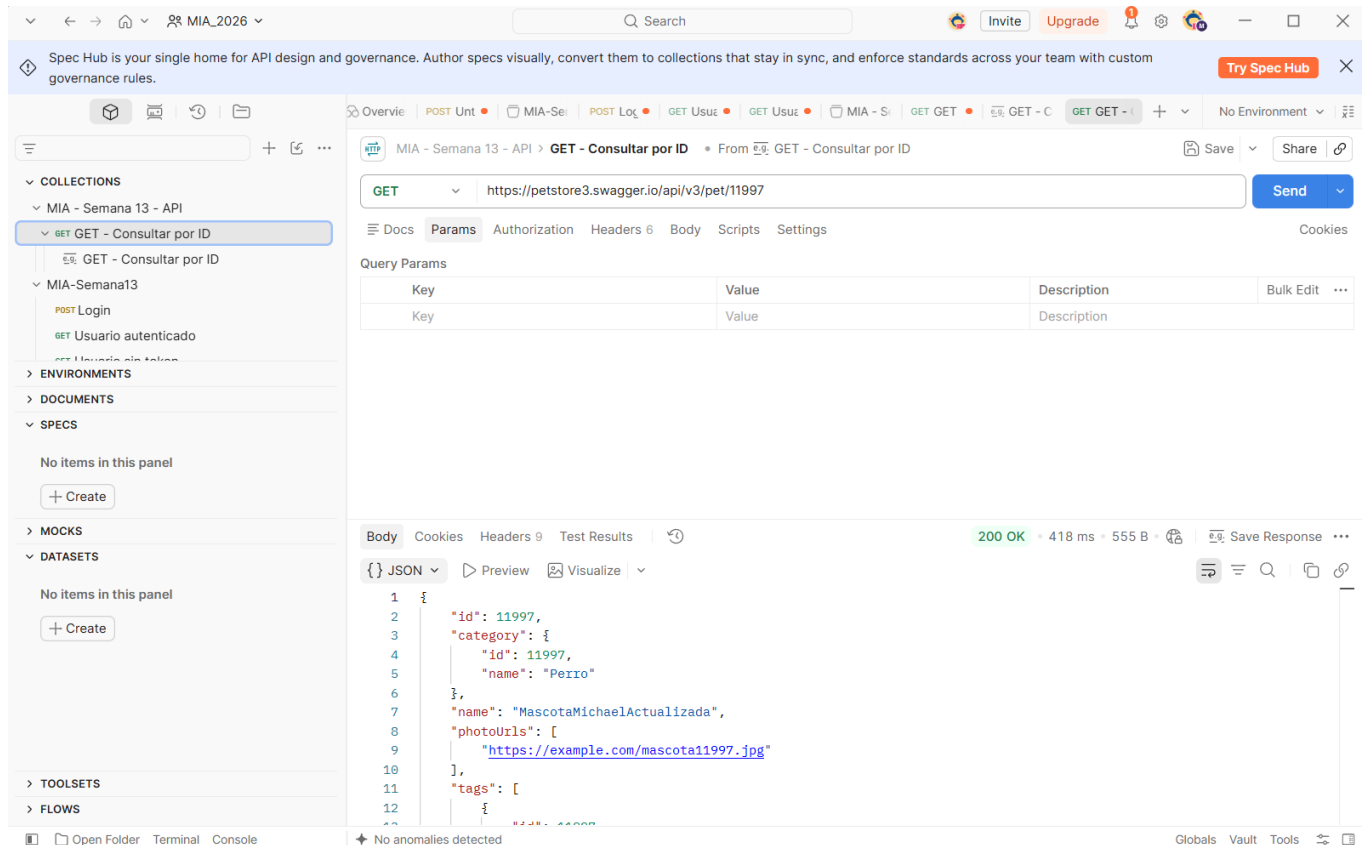
6. Reproduccion de las operaciones en Postman - Coleccion



Evidencia 9. Coleccion MIA - Semana 13 - API creada en Postman.

Se creo una coleccion independiente llamada **MIA - Semana 13 - API** para almacenar las solicitudes del laboratorio. Esta coleccion permite conservar las pruebas y ejecutarlas de forma organizada.

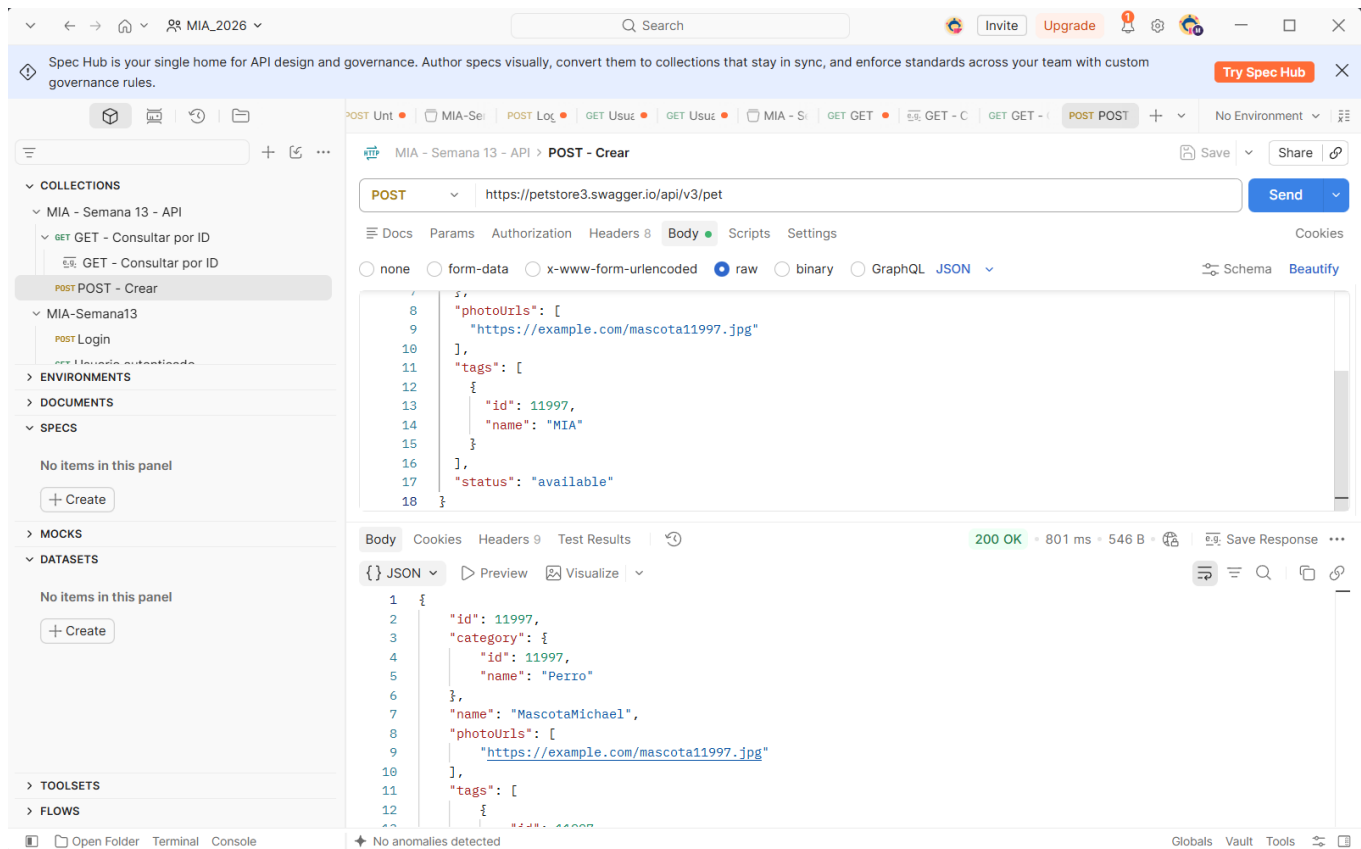
6.1 GET - Consultar por ID en Postman



Evidencia 10. Consulta de la mascota 11997 desde Postman.

Se configuro una solicitud **GET** hacia **`https://petstore3.swagger.io/api/v3/pet/11997`**. La respuesta fue **200 OK** y devolvio la informacion de **MascotaMichaelActualizada**.

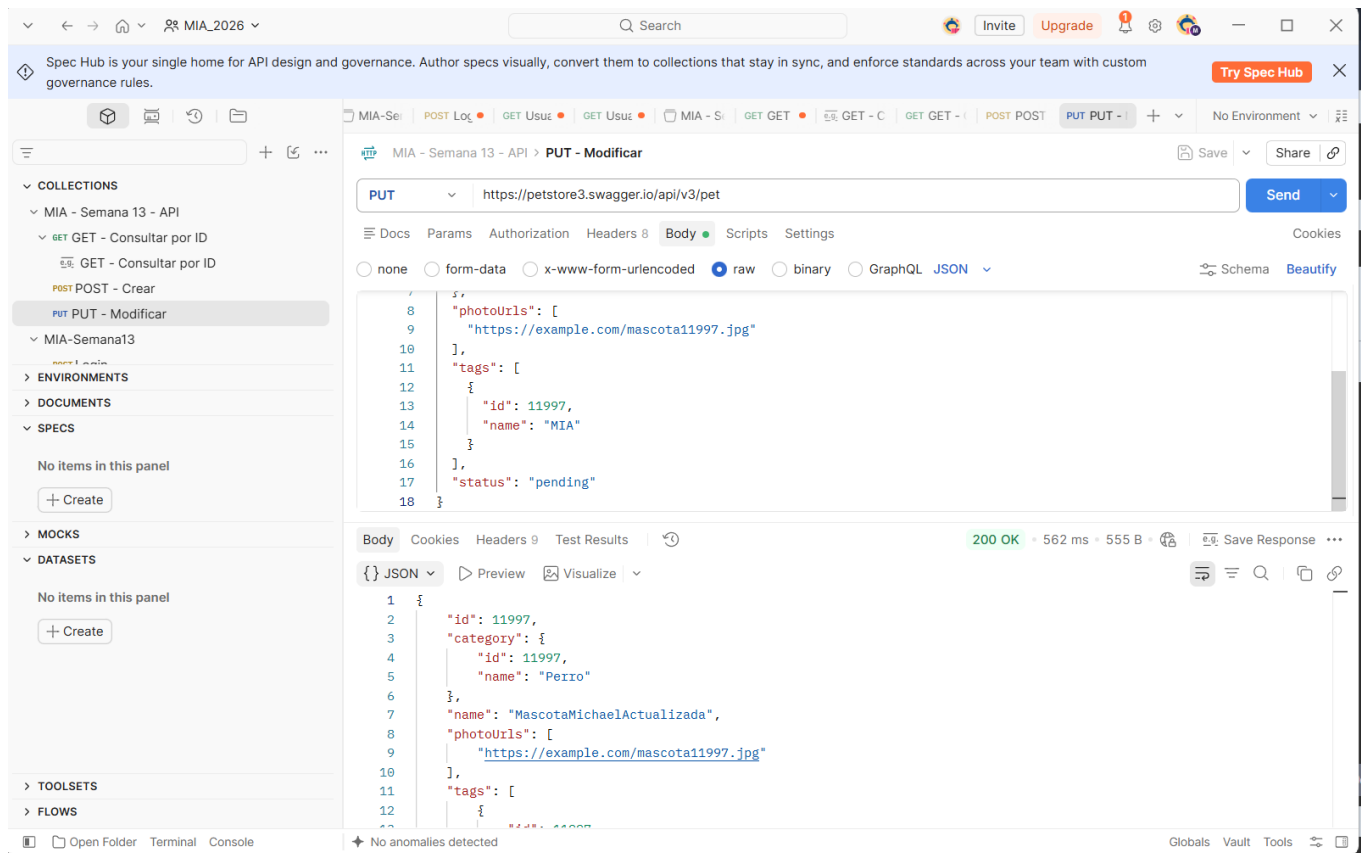
6.2 POST - Crear en Postman



Evidencia 11. Creacion de la mascota mediante POST desde Postman.

La solicitud **POST** se configuro con Body en formato **raw - JSON**. Se envio nuevamente la mascota con ID 11997 y estado available. Postman mostro **200 OK** y devolvio el recurso creado.

6.3 PUT - Modificar en Postman



Evidencia 12. Modificacion mediante PUT desde Postman.

Se utilizo **PUT** con el mismo endpoint base **/pet**. El JSON cambio el nombre a **MascotaMichaelActualizada** y el estado a **pending**. La respuesta obtenida fue **200 OK**.

6.4 GET - Buscar por status en Postman

The screenshot shows the Postman interface with a collection named 'MIA - Semana 13 - API'. The selected item is 'GET - Buscar por status'. The request method is 'GET' and the URL is 'https://petstore3.swagger.io/api/v3/pet/findByStatus?status=pending'. The 'Params' tab is active, showing a query parameter 'status' with the value 'pending'. The response is a 200 OK status with a response time of 644 ms and a body size of 13.52 KB. The response body is a JSON array containing one object representing a pet.

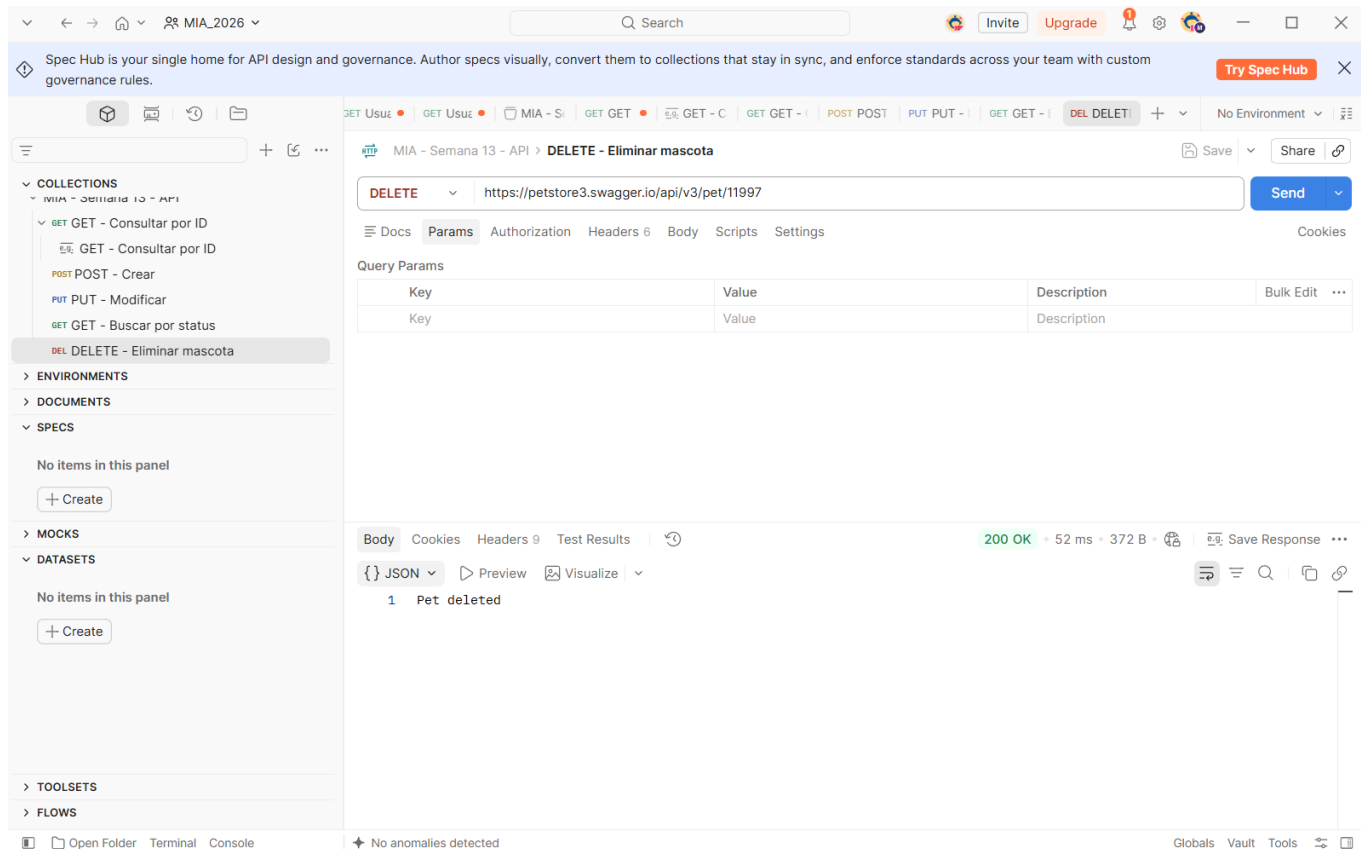
Key	Value	Description	Bulk Edit
status	pending		
Key	Value	Description	

```
1 [
2   {
3     "id": 100,
4     "category": {
5       "id": 10000,
6       "name": "Perros"
7     },
8     "name": "Kobe",
9     "photoUrls": [
10      "String"
11    ],
12    "tags": [
```

Evidencia 13. Consulta por status=pending desde Postman.

En la pestaña **Params** se agrego la clave **status** con valor **pending**. Postman construyo la URL con **?status=pending** y la API respondio con **200 OK** y un arreglo JSON.

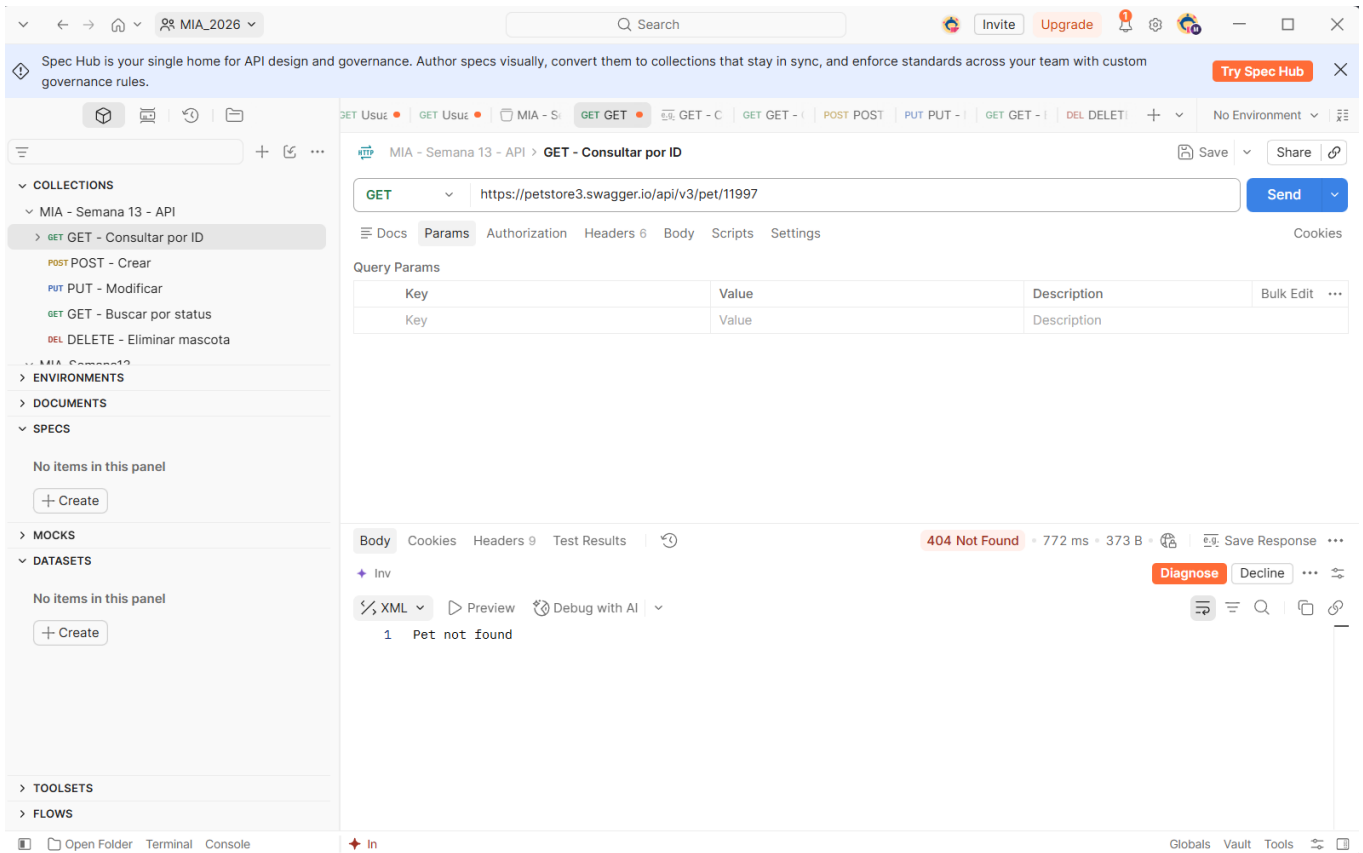
6.5 DELETE - Eliminar mascota en Postman



Evidencia 14. Eliminacion de la mascota 11997.

Se envio **DELETE** a `/pet/11997`. La API respondio con **200 OK** y el mensaje **Pet deleted**, confirmando la ejecucion de la operacion de eliminacion.

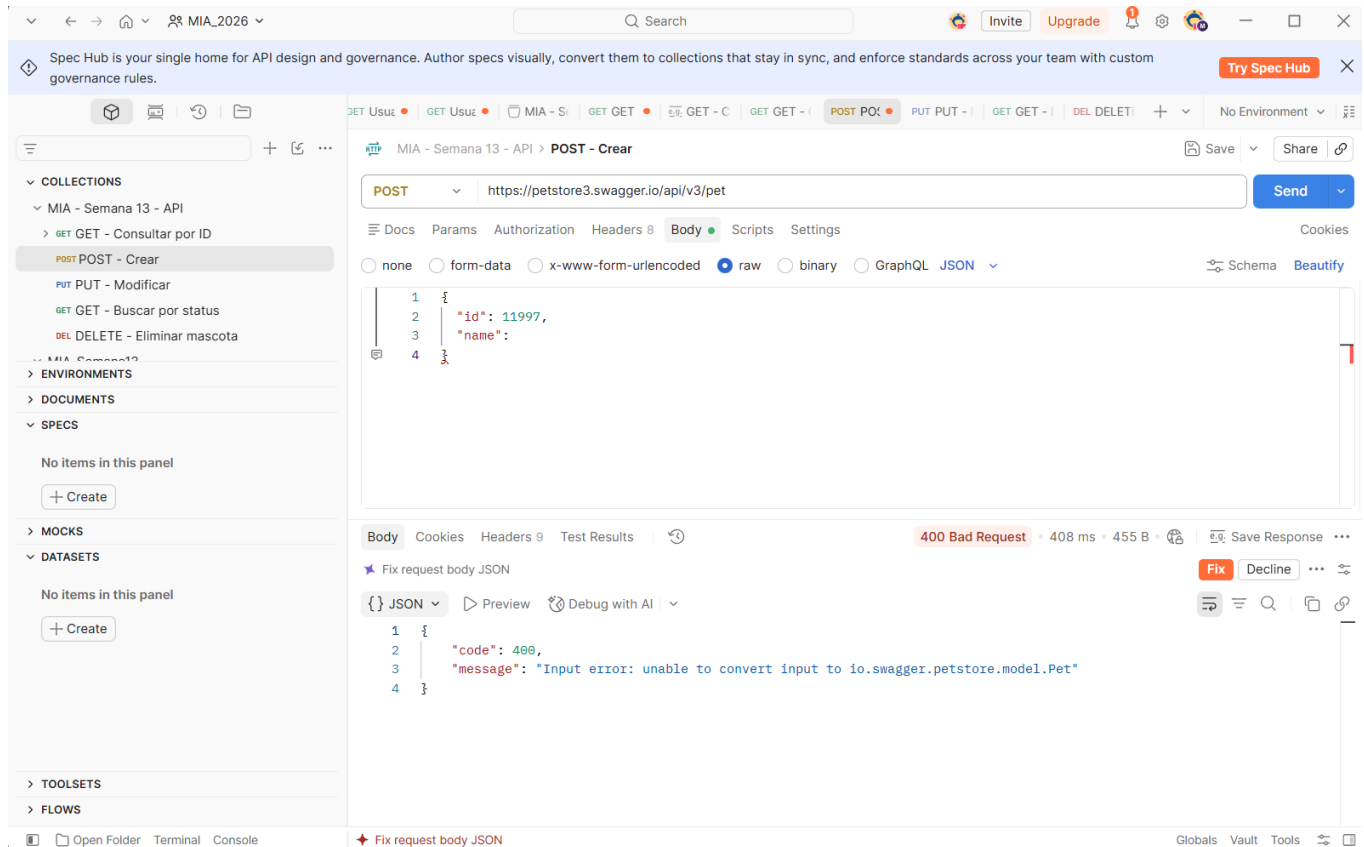
7. Prueba de errores - Recurso inexistente



Evidencia 15. Consulta de un recurso inexistente.

Despues de eliminar la mascota se ejecuto nuevamente **GET /pet/11997**. Se esperaba que el sistema indicara que el recurso no existia y la API respondio con **404 Not Found** y el mensaje **Pet not found**. La causa fue que el recurso habia sido eliminado previamente.

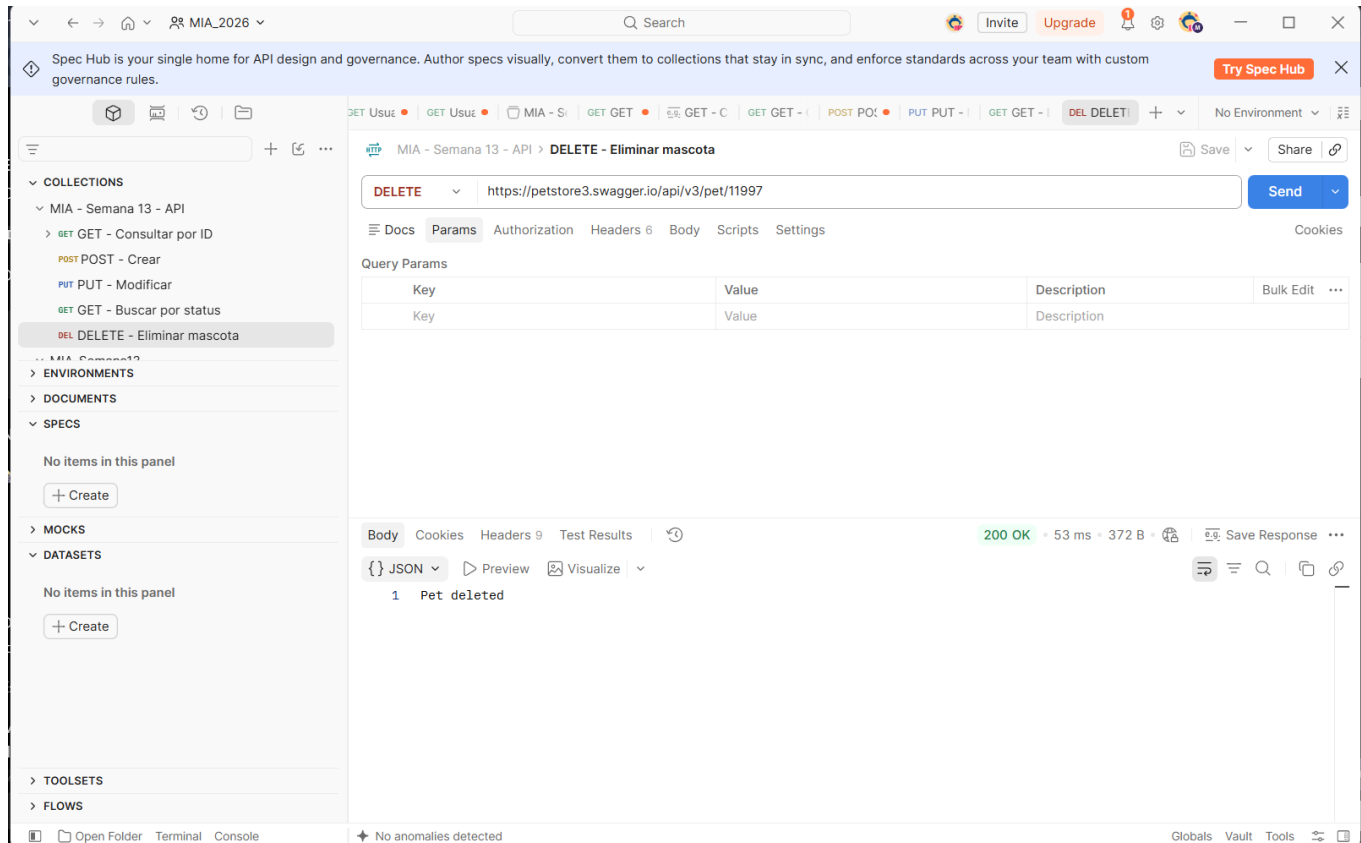
7.1 Prueba de errores - JSON incorrecto



Evidencia 16. Envio de un JSON mal formado.

Se envío intencionalmente un cuerpo JSON incompleto en una solicitud POST. Se esperaba que la API rechazara la solicitud y efectivamente respondió con **400 Bad Request**. El mensaje indicó que no fue posible convertir la entrada al modelo esperado. La causa fue la sintaxis inválida del JSON.

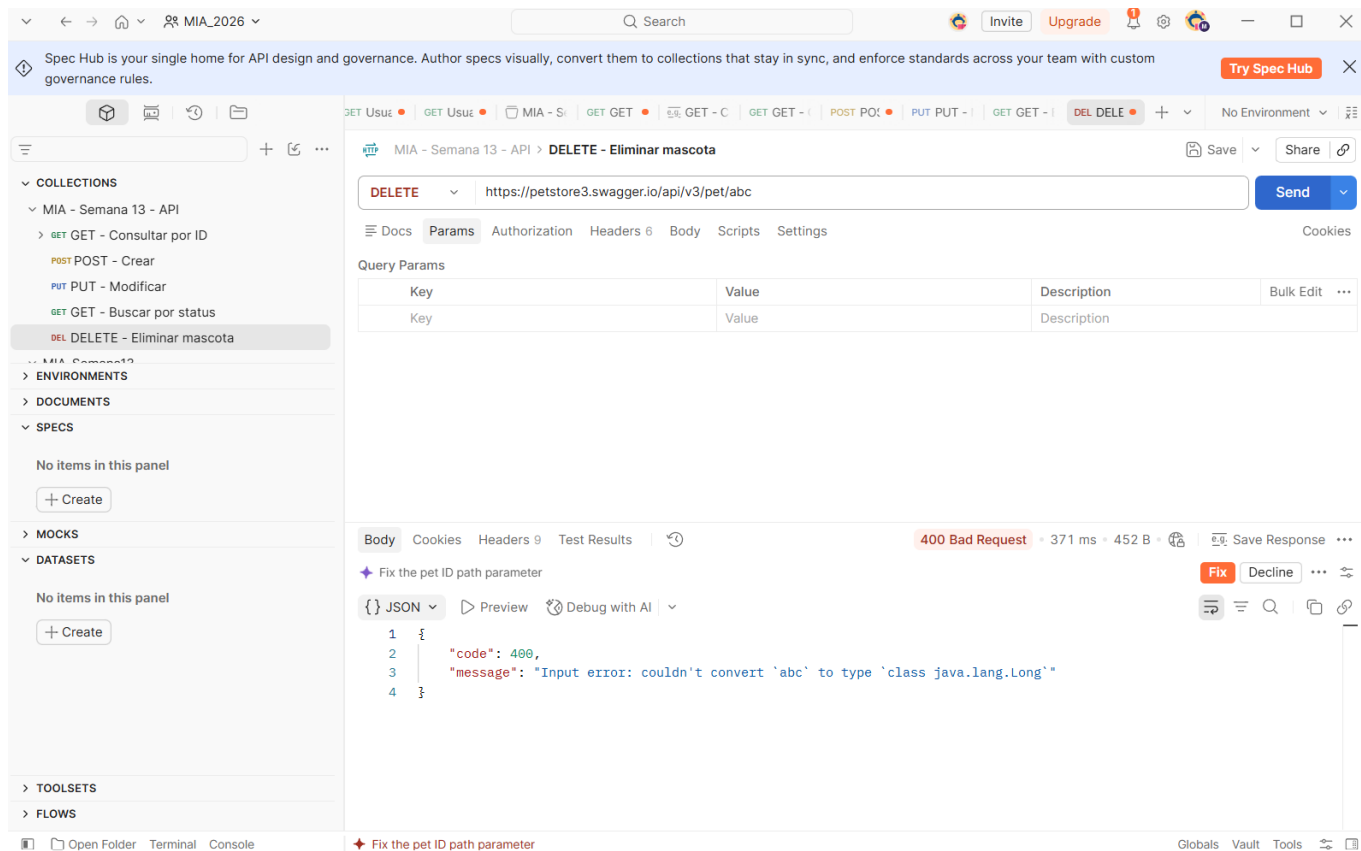
7.2 Prueba requerida - Eliminar nuevamente un recurso inexistente



Evidencia 17. Segundo intento de DELETE sobre /pet/11997.

La guía solicita intentar eliminar un recurso que ya no existe. Se ejecuto un segundo **DELETE /pet/11997** despues de haberlo eliminado y de haber comprobado con GET que devolvía 404. En esta implementacion publica de Petstore, el servidor respondio nuevamente con **200 OK** y **Pet deleted**. Por lo tanto, se registra el resultado real observado aunque no coincida con el error esperado para este escenario.

7.3 Prueba adicional - Identificador invalido



Evidencia 18. DELETE con un petId no numerico.

Como validacion adicional para obtener una tercera respuesta de error, se envio **DELETE /pet/abc**. Debido a que **petId** debe ser numerico, la API respondio con **400 Bad Request** y un mensaje indicando que no pudo convertir **abc** al tipo numerico esperado.

Resumen de las pruebas de error

Prueba	Esperado	Resultado	Causa / observacion
GET de recurso eliminado	Identificar que no existe	404 Not Found - Pet not found	El recurso 11997 ya habia sido eliminado.
POST con JSON incorrecto	Rechazo del cuerpo enviado	400 Bad Request	JSON incompleto y sintaxis invalida.
DELETE de recurso ya eliminado	Error por recurso inexistente	200 OK - Pet deleted	El servidor demo acepto nuevamente la eliminacion; se documento el comportamiento real.
DELETE /pet/abc	Error por ID invalido	400 Bad Request	El endpoint requiere un identificador numerico.

Las respuestas anteriores se registraron tal como fueron devueltas por la API durante las pruebas. En particular, el segundo DELETE sobre un recurso ya eliminado no produjo un error en el servidor de demostracion, por lo que se agrego una prueba con identificador invalido para documentar una tercera respuesta HTTP de error real.

8. API Detective

El flujo completo solicitado consiste en registrar una mascota, consultarla, actualizarla y eliminarla. Finalmente, el sistema debe identificar cuando el recurso ya no existe. Las evidencias obtenidas durante el laboratorio permiten comprobar cada etapa del flujo:

Paso	Metodo	Endpoint	Resultado observado
1. Registrar	POST	/pet	200 OK - Mascota 11997 creada.
2. Consultar	GET	/pet/11997	200 OK - Datos de la mascota recuperados.
3. Actualizar	PUT	/pet	200 OK - Nombre y estado modificados.
4. Eliminar	DELETE	/pet/11997	200 OK - Pet deleted.
5. Comprobar inexistencia	GET	/pet/11997	404 Not Found - Pet not found.

Endpoints y metodos definidos: POST /pet para registrar; GET /pet/{petId} para consultar; PUT /pet para actualizar; DELETE /pet/{petId} para eliminar; y GET /pet/{petId} nuevamente para verificar que el recurso ya no se encuentra.

El resultado final del flujo fue correcto: la mascota se creo, se consulto, se actualizo, se elimino y, al consultar nuevamente el ID 11997, la API devolvio **404 Not Found**. Esto permite identificar de forma clara la inexistencia del recurso.

Conclusion

El laboratorio permitio aplicar de forma practica el consumo de una API REST utilizando Swagger UI y Postman. Se identificaron endpoints, metodos HTTP, parametros, cuerpos JSON y codigos de estado. Tambien se ejecutaron las operaciones CRUD sobre un recurso de tipo mascota, se comprobaron modificaciones mediante consultas posteriores y se analizaron respuestas de error.

Swagger facilito la exploracion y documentacion de los endpoints, mientras que Postman permitio reproducir las mismas operaciones de forma organizada dentro de una coleccion. Las pruebas demostraron la importancia de revisar tanto el codigo HTTP como el cuerpo de la respuesta, ya que una API publica de demostracion puede presentar comportamientos particulares, como aceptar una segunda eliminacion de un recurso. En conjunto, la actividad permitio comprender el flujo completo de comunicacion con una API y la forma de validar sus respuestas antes de integrarla en una aplicacion.